

CME 213 – Milestone 4

CUDAsearch: Distributed GPU-Accelerated Dense Vector Similarity Search

Ava Kouhana · Ethan Cohen

1 Summary of progress

Since Milestone 3, we have implemented the distributed multi-GPU version of CUDAsearch using MPI. Each MPI rank manages one GPU and owns a row-shard of the database; the single-GPU CUDA kernels from M3 (`tilted FP32` and `int8_tiled`) are reused. We collected strong-scaling data on SIFT1M and GIST1M at 1, 2, and 4 ranks, weak-scaling data on SIFT1M, and Nsight Systems timelines at 1 and 4 ranks. Top- k selection still runs on the CPU at each rank after a host copy of the local score matrix. We identified this at M3 as the largest remaining single-GPU cost, and it now also limits multi-rank scaling on SIFT1M.

2 Distributed algorithm and data decomposition

The database $X \in R^{N \times d}$ is split by rows across P ranks; rank r owns approximately N/P contiguous rows, denoted $X^{(r)}$, with global offset $o_r = \sum_{s < r} |X^{(s)}|$. The query batch $Q \in R^{B \times d}$ is replicated on every rank.

Each search call is bulk-synchronous. The root broadcasts Q (≈ 50 KB on SIFT1M) via `MPI_Bcast`; every rank then independently computes $S^{(r)} = Q(X^{(r)})^\top$, runs a host top- k on its local scores, shifts indices by o_r , and ships its $B \cdot k$ (index, score) pairs to rank 0 with two `MPI_Gatherv` calls ($P \cdot B \cdot k$ pairs total); rank 0 then performs the final merge. The only setup-time exchange is a one-shot `MPI_Scatterv` of X at index build, amortised across all subsequent queries.

We considered a 2D decomposition (split both queries and database) but rejected it because at $B = 100$ and $P = 4$ the per-rank kernel becomes too small to amortise launch overhead and host staging. We also rejected a fully replicated database (each rank searches everything and picks every P -th query) because it leaves per-rank memory unchanged and only divides the score-matrix work, which does nothing for the $B \times N$ D2H plus host top- k cost identified in our Nsight profile.

3 MPI implementation

Each search call uses three collectives. `MPI_Bcast` ships the query batch Q from rank 0 to all ranks at the start of the call; ranks then run their local matmul and host top- k independently; rank 0 collects each rank's local candidate count with `MPI_Gather`, followed by two `MPI_Gatherv` calls (indices and scores) carrying the $B \cdot k$ pairs per rank. Rank 0 finalises the result with a P -way merge per query using `std::nth_element + std::sort`, total cost $O(B \cdot k \cdot (P + \log k))$ — negligible at the scales tested. We use `Gatherv` rather than `Gather` to tolerate the case `local_k < k` when a shard is smaller than k rows; this never triggers at $N = 10^6$, $P \leq 4$ but keeps the dispatch robust. No point-to-point calls are used.

Computation and communication are not overlapped: ranks synchronise at batch boundaries, copy the local $B \times N$ score matrix to host, run CPU top- k , then enter the gather. GPU buffers are staged through host memory rather than CUDA-aware MPI because the only object crossing the network is the small top- k payload, which already lives on the host after CPU selection; the much larger per-rank score matrix never leaves the node.

4 C++ wrapper and integration with CUDA

At startup each rank calls `MPI_Init`, derives its node-local rank via `MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)`, and binds to one GPU with `cudaSetDevice(local_rank % ndev)`. Each rank then invokes the same single-GPU kernels from M3 over its local row-shard $X^{(r)}$; the kernels are byte-for-byte unchanged, and the only host-side adaptation is shifting local indices by the global row offset o_r before the gather. Rank 0 owns the global top- k output buffers and runs the final merge on the CPU.

The INT8 tiled path is wrapped by a persistent `GpuInt8TiledIndex` object built once at setup, which uploads and quantises $X^{(r)}$ to device memory and reuses its scratch buffers across calls. The FP32 tiled path retains M3’s per-call `cudaMalloc` / `cudaMemcpy` / `cudaFree` lifecycle, so $X^{(r)}$ is re-uploaded each batch (≈ 47 ms per rank on SIFT1M, visible in the Nsight summary).

5 Correctness testing

The `test_mpi` CTest target builds a small synthetic database ($N = 4096$, $d = 128$, $B = 64$, $k = 10$) with a known ground truth and runs the MPI driver at 1, 2, and 4 ranks against both the FP32 tiled and INT8 tiled backends. For each rank count and backend the test asserts (i) that the distributed result agrees with the single-GPU reference at $\geq 99.9\%$ recall and (ii) that the recall against the synthetic ground truth clears 0.999 for FP32 and 0.98 for INT8. On the real datasets, end-to-end Recall@10 is ≈ 0.989 – 0.990 across rank counts for FP32 tiled on SIFT1M and 0.9630 for INT8 tiled, matching the single-GPU baselines. On GIST1M, Recall@10 is 0.3560 at every rank count (the low value is a normalisation / ground-truth artefact carried over from M3, not a correctness regression).

6 Performance, scaling, and profiling

All benchmarks were run on the Stanford HPC node `hpcc-gpu-5-1` (one Quadro RTX 6000, sm_75; 16 CPU cores per node shared across ranks), with $N = 10^6$, $B = 100$, $k = 10$. Tables 1 summarise the headline numbers; full CSVs are in `results/mpi/`.

Dataset	Ranks	Latency (ms)	QPS	Speedup	Efficiency
SIFT1M	1	743	134.6	1.00×	100.0%
SIFT1M	2	473	211.6	1.57×	78.6%
SIFT1M	4	246	407.1	3.03×	75.6%
GIST1M	1	1482	67.5	1.00×	100.0%
GIST1M	2	890	112.4	1.67×	83.3%
GIST1M	4	386	259.2	3.84×	96.1%

Table 1: Strong scaling of the FP32 tiled MPI kernel, $N = 10^6$, $B = 100$, $k = 10$.

GIST1M scales better than SIFT1M because its dimension is 960 rather than 128, so each candidate dot product has 7.5x more arithmetic while the top- k candidate communication is unchanged. The SIFT1M gather phase grows from about 0.04 ms at 1 rank to 15.6 ms at 4 ranks, but local search still dominates total latency.

6.1 Nsight Systems profile of the FP32 tiled kernel

We captured one search invocation at 1 rank and 4 ranks with Nsight Systems (SIFT1M, $N = 10^6$, $B = 100$, $k = 10$). Table 2 gives the phase-level breakdown from `nsys stats`.

Phase	1-rank (ms)	4-rank, per rank (ms)	Scaling
<code>gpu_search_tiled_kernel</code> (GPU)	21.87	5.47	$4.00\times$
D2H memcpy ($B \times N$ score matrix)	41.21	11.39	$3.62\times$
H2D memcpy ($X^{(r)}$, per call)	46.92	12.18	$3.85\times$
<code>cudaLaunchKernel</code>	0.20	0.22	$0.91\times$
GPU/CUDA-API subtotal	≈ 110	≈ 29	$3.79\times$
Host top- k (CSV <code>local_ms</code> – subtotal)	≈ 633	≈ 216	$2.93\times$
End-to-end <code>local_ms</code> (CSV, 5-trial mean)	743	245	$3.03\times$

Table 2: Per-phase costs from Nsight Systems on a single search call, compared against the CSV-reported `local_ms`. The current FP32 path re-allocates and re-uploads $X^{(r)}$ on every call; the INT8 tiled path keeps a persistent device-side index and avoids this cost.

From the Table 2, we infer that main scalability bottleneck is therefore not kernel launch overhead or query transfer. It is the design choice to materialize the full $B * (N/P)$ score matrix, copy it back to the host, and run CPU `nth_element` top- k for every query. Communication is secondary at 4 ranks, though `MPI_Gatherv` and rank imbalance are starting to show up in the SIFT1M and GIST1M timing breakdowns.

7 Discussion of approaches and bottlenecks

The most effective design choice in M4 was row-sharding the database across ranks. It reused the M3 CUDA kernels unchanged while giving a $4.5\times$ speedup over the 16-core CPU baseline and a $3.0\times$ speedup over the single-GPU FP32 tiled kernel. The main remaining bottleneck is inherited from M3: each rank still materialises a $B \times (N/P)$ FP32 score matrix, copies it to host, and runs CPU top- k . At $P = 4$ on SIFT1M this is still about 100 MB per rank per batch. A GPU-resident top- k would reduce the transfer to only $B \times k$ candidate pairs, a 2.5×10^4 reduction, and remove the large host `nth_element` cost seen in Nsight.

CUDA-aware MPI and asynchronous gather were not worth adding yet because only $B \cdot k$ candidate pairs per rank cross the network. The scaling limit depends on the dataset: SIFT1M is already mostly host-top- k /D2H limited at 4 ranks, while GIST1M reaches 96.1% efficiency because its $d = 960$ dot products provide much more GPU work per communicated candidate. Once GPU top- k removes the host bottleneck, `MPI_Gatherv` will become the next likely scaling limit, especially beyond roughly 8 ranks unless larger batches amortise collective latency.

8 Challenges and next steps

The main implementation challenge was making the merge correct under uneven shards and verifying it across rank counts; `Gatherv` plus a small CPU heap turned out to be enough. For the final report (Jun 8) we plan to implement GPU-resident top- k to eliminate the score-matrix copy, benchmark it, re-run strong scaling... This is the single most important remaining optimization. If we can, we will also add a comparison against FAISS `GpuIndexFlatIP` as an external upper bound.